

Claude Code skills for Extreme Networks labs on GNS3

Switch Engine · Fabric Engine · Site Engine — guide and full skill text

Assumes	Only the IP address of a working GNS3 server (API v2)
Contents	8 skills: GNS3 API, switch templates, Switch Engine, Fabric Engine, simulated devices, Site Engine, FlexViews, product documentation
Validated on	GNS3 2.2.45 · EXOS VM 32.6.3.126 · VOSS VM 9.4.0.0 · XIQ-SE 24.2.15.5 · Linux clients
Author	Salva Ferrer (AVTN) · community contribution · 2026-09-30
License	Text CC BY 4.0 · scripts MIT

About this guide

This document goes with the `extreme-gns3-skills` pack. It explains how to use the pack with Claude Code and reproduces the full text of every skill so a person can review it before installing. The scripts are only in the zip.

Who it is for: engineers, trainers and partners who build labs with Extreme Networks virtual products on GNS3 and want Claude Code to do the repetitive work: creating templates from the official GitHub images, building topologies through the API, booting and configuring switches through their consoles, backing up configurations, and bringing Site Engine into the lab.

The only assumption: the IP address of a working GNS3 server (`<GNS3_HOST>`) whose API port (3080) and console ports (5000-9999) you can reach. Everything else (templates, images, projects) can be created from there.

How to work with Claude Code on a lab

- **Plan first, then execute.** For anything beyond a quick check, let Claude investigate and propose a plan (Claude Code's *plan mode*), approve it, then let it run. Say whether it may run the whole plan unattended or must stop at checkpoints.
- **Shared server, explicit consent.** On a GNS3 server used by others, ask Claude to confirm before anything that changes shared state: starting or stopping the server, opening or closing other projects, modifying templates or images in use. Inside its own throwaway project it can follow an approved plan.

- **Evidence over assumptions.** Ask Claude to check every claim against the device: command output, `show` commands, the API's own view (`GET .../nodes`, `command_line`), and the official documentation of the exact version (`extreme-product-docs`).
- **Consoles are shared.** Whatever Claude types on a switch console is visible to anyone attached to it, and vice versa. Never experiment on a switch someone else is using.
- **Credentials** go in environment variables or are typed by the human, never in files or chat.
- **Save what you learn.** Findings from each session (a gotcha, a working command) should go into project notes or into these skills, so the next session starts from there.

Quick start: from an IP address to a small lab

1. "Check the GNS3 server at `<GNS3_HOST>` and list its templates." (`gns3-server-api`)
2. "Add Fabric Engine 9.4.0.0 and Switch Engine 33.6.1.14 from Extreme's GitHub as templates." (`extreme-switch-templates`: download, md5 check, upload through the API, template from the `.gns3a`)
3. "Create a project `my-lab` with two Fabric Engine switches linked on 1/1 and one Switch Engine on 1/24 of the second, start them." (`gns3-server-api`)
4. "Do the first login on each switch and show me LLDP neighbors and IS-IS adjacencies." (`fabric-engine-in-gns3`, `switch-engine-in-gns3`)
5. "Save and back up all configs to `configs/v1`." (`backup_configs.py`)
6. "Add a FreeRADIUS container and an 802.1X client on port 2, and an FRR router peering OSPF on port 3." (`simulated-devices-in-gns3`)
7. Optional: "Add Site Engine to the management network and discover the switches." (`site-engine-in-gns3`)

What has been validated

Product	Version	Status
GNS3 server	2.2.45 (API v2)	Validated. GNS3 3.x (API v3) not tested.
Switch Engine (EXOS VM)	32.6.3.126	Validated in multi-switch labs over several months.
Fabric Engine (VOSS VM)	9.4.0.0	Validated in a live test from GitHub to a two-node fabric. Fabric services not tested.
Site Engine (XIQ-SE)	24.2.15.5	Validated as a GNS3 VM with EXOS VM switches.
Simulated devices (Alpine)	alpine:latest	LLDP, 802.1X (EAP-MD5) + FreeRADIUS, FRR OSPF, 802.1Q hosts validated against EXOS VM; LAG from a container did not work; others not validated.
ExtremeControl, ExtremeAnalytics	—	Not validated (virtual appliances were not available). Guidance only.

Clients were Linux only. Each skill ends with an **environment dependencies** table that says what to look for if your setup differs. The detailed evidence is in the appendix "Validation log".

Giving Claude the product documentation

Claude's general knowledge of Extreme CLIs is not version-exact. Download the User Guide, Command Reference, EMS Message Catalog (and for Site Engine, its documentation collection) **for the versions in your lab**, put them in a `docs/` folder, run `scripts/prepare_docs.py docs/` (PDF and HTML to searchable text, plus an index), and tell Claude the folder exists. The skill `extreme-product-docs` makes Claude search it before stating a command, prefer the live CLI when they disagree, and cite the page it used.

Installing and using the pack

A skill is a folder with a `SKILL.md`. Copy the folders under `skills/` to `~/.claude/skills/` (all your projects) or to `.claude/skills/` inside a project. Claude Code lists them at start-up and uses one when the task matches its description; you can also name it explicitly ("use the fabric-engine-in-gns3 skill"). Scripts need Python 3 (standard library only); `prepare_docs.py` also uses `pdftotext`.

The scripts are referenced with paths relative to each skill folder (e.g. `../gns3-server-api/scripts/gns3_console.py`): keep the folders side by side.

Good practice

- Work in **your own GNS3 project**; never touch other people's projects on a shared server.
- Test on throwaway nodes, never on a switch someone is using: consoles are shared.
- Keep credentials out of files and chats: the scripts read `CONSOLE_USER`/`CONSOLE_PASS` from the environment.
- The Extreme virtual images are not supported by GTAC and have their own license (see each GitHub repo): this pack links to them and never redistributes them.

Skill `gns3-server-api`

Use when driving a remote GNS3 server (API v2) to build or operate a lab with Extreme Networks virtual devices (Switch Engine/EXOS, Fabric Engine/VOSS, Site Engine) given only its IP address — checking the server, projects, templates, nodes, links, consoles, NAT, and the known API pitfalls (phantom ports, adapter resize breaking links, `auto_close`, console port range). For adding switch models see `extreme-switch-templates`.

The only assumption is `<GNS3_HOST>`: the IP (or name) of a working GNS3 server whose API port (default 3080) and console port range are reachable from where Claude Code runs. No GUI is needed; everything here is plain HTTP + JSON. Replace placeholders in `<ANGLE_BRACKETS>`.

Validated environment

- GNS3 server **2.2.45**, API **v2**, no API authentication, one compute (`local`), KVM available on the host. Clients: Linux, `curl` 8.x, Python 3 standard library.
- Devices: Switch Engine (EXOS VM) 32.6.3, Fabric Engine (VOSS VM) 9.4.0, Site Engine (XIQ-SE) 24.2 as a QEMU VM, Linux QEMU VMs, docker (Alpine), NAT and `ethernet_switch` nodes.
- **GNS3 3.x uses API v3** (different paths, token authentication). The concepts below still hold, but check the v3 documentation before reusing any call.

Ground rules

1. **Check before anything else:** `curl -s -m 5 http://<GNS3_HOST>:3080/v2/version`. No answer means no lab: fix reachability (VPN, firewall, server powered off) first. A server that has just booted needs a minute or two before the API answers.
2. **Work in your own project.** A GNS3 server is often shared (courses, other engineers). Never open, close, modify or delete a project you did not create: closing a project stops all of its nodes. Ask the human before touching anything shared (templates in use, images).
3. **Test in a throwaway project first**, never on nodes someone is using. A telnet console is shared: whoever is attached sees everything you type, including `show` commands.
4. Before creating big things (templates, many VMs) check free disk on the server if you have a shell there (images live under `/opt/gns3/images/` in a typical install).

Core API calls (v2)

```
GET /v2/version # server version
GET /v2/computes # usually just "local"
GET /v2/projects # name, project_id, status (opened/closed)
POST /v2/projects {"name": "MY_LAB", "auto_close": false}
POST /v2/projects/<p>/open | /close
PUT /v2/projects/<p> {"auto_close": false}
GET /v2/templates # name, template_id, template_type
POST /v2/projects/<p>/templates/<t> {"x": 0, "y": 0, "compute_id": "local", "name": "SW1"}
GET /v2/projects/<p>/nodes # status, console, console_type, ports[]
POST /v2/projects/<p>/nodes/<n>/start | /stop | /reload
PUT /v2/projects/<p>/nodes/<n> {"name": ..., "properties": {...}} # node STOPPED
DELETE /v2/projects/<p>/nodes/<n>
GET /v2/projects/<p>/links
POST /v2/projects/<p>/links {"nodes": [{"node_id": A, "adapter_number": 1, "port_number": 0},
                                {"node_id": B, "adapter_number": 1, "port_number": 0}]}
DELETE /v2/projects/<p>/links/<l>
GET /v2/compute/qemu/images # QEMU images on the server
POST /v2/compute/qemu/images/<filename> # upload an image (raw body)
POST /v2/templates # create a template (extreme-switch-templates)
```

Example session:

```
H=http://<GNS3_HOST>:3080/v2
P=$(curl -s -X POST $H/projects -H 'Content-Type: application/json' \
-d '{"name":"my-lab","auto_close":false}' | python3 -c 'import
json,sys;print(json.load(sys.stdin)["project_id"])')
curl -s $H/templates | python3 -c 'import json,sys;[print(t["template_id"],t["name"]) for t in
json.load(sys.stdin)]'
curl -s -X POST $H/projects/$P/templates/<TEMPLATE_ID> -H 'Content-Type: application/json' \
-d '{"x":0,"y":0,"compute_id":"local","name":"SW1"}'
curl -s $H/projects/$P/nodes | python3 -c '
import json,sys
for n in json.load(sys.stdin): print(n["name"], n["status"], n["console_type"], n["console"])'
```

- **Always pass "compute_id": "local"** when creating nodes and templates: some creations (NAT node, docker template) fail without it.
- **Ports: never guess the mapping.** Read `ports[]` of the node (`adapter_number`, `port_number`, `name`). For Extreme appliances adapter 0 is `Mgmt`, and adapter `n` is data port `n` (EXOS `Port<n>`, VOSS `1/<n>`); VOSS adapters 25-32 are the breakout ports `1/25/1...1/26/4`.
- **Set `auto_close: false`** on your project. Otherwise the server closes the project, and stops every node in it, when it sees no GUI client connected, which is always the case for API-only work. Misleading symptom: consoles or remote desktops "unreachable" in a loop, while in fact the whole project is closed (`GET /v2/projects/<p> → status`).
- **NAT to the outside world:** there is no invocable "NAT" template; create the node directly: `POST /v2/projects/<p>/nodes` with `{"name": "NAT1", "node_type": "nat", "compute_id": "local", "properties": {"ports_mapping": [{"interface": "vibr0", "name": "nat0", "port_number": 0, "type": "ethernet"}]}}`. It gives DHCP in `192.168.122.0/24` and masquerades **only that subnet**: traffic routed from other subnets behind it is lost. Put a Linux router (with its own NAT) in between if several lab subnets need Internet access.

- `ethernet_switch` nodes are cheap, have no CLI and are ideal as fan-in segments (management network, services network). Create them large (e.g. 48 ports) from day one.

Consoles

- `console_type` `telnet` (switches) → `telnet <GNS3_HOST> <console>` or, from a script, `scripts/gns3_console.py` (below). `vnc/spice` (VMs) → a VNC/SPICE viewer on the same port. **The console port of VMs may change between restarts:** always read it from the API.
- The GNS3 console port range is **5000-9999** by default. A manual `console` value outside it is silently ignored and replaced. If you design a port numbering scheme, keep it within 4 digits. Changing `console` needs node stopped → `PUT` → start (does not affect links).
- If a remote-desktop gateway (e.g. Apache Guacamole) will front the VMs, use `vnc`, not `spice`: the official `guacd` image has no SPICE support. Test with a banner check (`cat < /dev/tcp/<GNS3_HOST>/<port>` prints `RFB 003.008` for VNC), not only `nc -z`.

`scripts/gns3_console.py`:

```
python3 scripts/gns3_console.py <GNS3_HOST> <port> cmd "show version" # EXOS
CONSOLE_USER=rwa CONSOLE_PASS='<new password>' python3 scripts/gns3_console.py <GNS3_HOST>
<port> \
    cmd "enable" "terminal more disable" "show isis adjacencies" # VOSS
python3 scripts/gns3_console.py <GNS3_HOST> <port> cmd "save" # then:
python3 scripts/gns3_console.py <GNS3_HOST> <port> answer y # reply to the y/N prompt
```

It logs in if asked, waits for the prompt after every command (so long outputs are complete), and supports `--wizard`, `esc`, `ctrl-c` and `vi-upload` (see its header).

Console gotchas seen on both Switch Engine and Fabric Engine:

- **Send `\r`, not `\r\n`:** `\r\n` counts as two lines, so a password gets an extra empty answer and the login fails (three failures lock the VOSS console for 60 s).
- Interactive steps (first-boot wizards, forced password change, y/N confirmations) are safest driven one prompt at a time: send, wait for the exact prompt text, send the next. Log messages printed on the console (VOSS prints many while it boots: ZTP+, cloud agent, Auto-Sense) can hide the prompt from a naive "last line" check.
- A telnet console session survives disconnects: once logged in, the next connection lands at the same prompt (and so does anyone else's).

Known API pitfalls (and how to avoid them)

- **`PUT` changes to `adapters`, `adapter_type`, `ports_mapping`, `console_type` or `options` are not applied to a running node.** Always stop → `PUT` → start.
- **Resizing `adapters` on a node that already has links can break them.** Observed: all UDP tunnels of the node were reassigned, links stayed listed in `/links` but came back with `N0-CARRIER`; restarting the peers did not help; the only fix was deleting and recreating the links (and, for a single-port NAT node, the whole node). Changing `console_type` or adding QEMU devices can also reorder the VM's PCI bus, so a Linux guest renames its NICs (`ens4` → `ens3`). **Design rule: never resize, over-provision.** Create each template/node with the adapter count it will ever need (plus a spare or two) in the creating `POST`; later "add an interface" becomes "link an adapter that already exists", which never

triggers the bug. For Linux guests, pin NetworkManager profiles by MAC, not by interface name:

```
nmcli connection modify '<profile>' 802-3-ethernet.mac-address <MAC> connection.interface-name ''.
```

- **Phantom ports:** after deleting a link, the port may stay reserved ("Port X isn't free" although `/links` doesn't list it). Restarting the node does not free it. Use another free port; for a single-port node (`nat`), delete and recreate the node (it has no state).
- After any resize or console change, re-check `GET .../links` before assuming the cabling is unchanged.

Optional: if you also have a shell on the GNS3 host

Not required by the rest of the pack, but useful:

- **QEMU guest agent** (run commands inside a Linux VM without network): add to the node's `properties.options` `-device virtio-serial-pci -chardev socket,path=/tmp/qga-<node>.sock,server,nowait,id=qga0 -device virtserialport,chardev=qga0,name=org.qemu.guest_agent.0` (stop → PUT → start; one **unique** socket per node, never at template level or two nodes collide), install `qemu-guest-agent` in the guest, then on the host: `echo '{"execute":"guest-exec","arguments":{"path":"/bin/hostname","capture-output":true}}' | sudo nc -U /tmp/qga-<node>.sock` and read the result with `guest-exec-status + {"pid": N}` (out-data is base64). Feed the JSON through stdin, never interpolated into a quoted shell string.
- Disk: `df -h` of the images/projects filesystem before creating templates. Do large `qemu-img` work in a scratch directory on that big filesystem, not in `/tmp` (often on a small root partition).

Environment dependencies — what to look for in yours

Need	What we used	What to identify in your environment
Reach the API and console ports	Routed/VPN access to <code><GNS3_HOST></code>	How you reach 3080 and 5000-9999 without re-routing all your traffic
HTTP + JSON client	<code>curl</code> , Python 3 <code>stdlib</code>	Any; Python <code>stdlib</code> is enough for the scripts
Console client for scripts	<code>gns3_console.py</code> (sockets)	Only Python 3 and network access to the console port
API version	v2 (GNS3 2.2)	GNS3 3.x → API v3, adapt paths and auth

Skill extreme-switch-templates

Use when adding an Extreme Networks switch model or firmware version to a remote GNS3 server (Switch Engine/EXOS from github.com/extremenetworks/Virtual_EXOS, Fabric Engine/VOSS from [Virtual_VOSS](https://github.com/extremenetworks/Virtual_VOSS)), creating a GNS3 template from a .gns3a appliance file without the GUI, or modifying an existing switch template (more ports, RAM, console, QEMU options such as `-cpu host`). Needs only the GNS3 server IP.

Only `<GNS3_HOST>` is assumed (see [gns3-server-api](#) for the API basics). Everything here works through the API: no GUI and no SSH to the server are needed.

Validated environment

- GNS3 server 2.2.45, API v2. Linux client, Python 3 standard library.
- **Fabric Engine 9.4.0.0**: full path from GitHub to a running two-switch fabric, done with `scripts/gns3a_to_template.py` (see `VALIDATION.md` of the pack).
- **Switch Engine 32.6.3.126**: official 13-port template and a 24-data-port extension, used in multi-switch labs. Newer EXOS versions (32.7.2.19, 33.x) use the same procedure but were not all turned into templates in our lab.

1. Official sources (github.com/extremenetworks)

Product	Repo	Appliance file	Images (qcow2)
Switch Engine (EXOS)	<code>Virtual_EXOS</code>	<code>exosvm.gns3a</code>	EXOS-VM_ <code><ver></code> .qcow2 (32.6.3.126 has a <code>v</code> prefix: EXOS-VM_v32.6.3.126.qcow2); also <code>.iso</code> (install) and <code>.xos</code> (in-place upgrade)
Fabric Engine (VOSS)	<code>Virtual_VOSS</code>	<code>VOSSGNS3-V9.X.gns3a</code> (also <code>.8.X</code> , <code>.7.X</code>)	FEGNS3. <code><ver></code> .qcow2 (9.x), VOSSGNS3. <code><ver></code> .qcow2 (8.x)

Images are served from https://akamai-ep.extremenetworks.com/Extreme_P/github-en/<repo>/<file>. The `.gns3a` lists every version with its file name, **md5** and download URL:

```
curl -s0 https://raw.githubusercontent.com/extremenetworks/Virtual_VOSS/master/VOSSGNS3-V9.X.gns3a
python3 scripts/gns3a_to_template.py VOSSGNS3-V9.X.gns3a --list
```

Key appliance parameters (both use `-cpu host`, `telnet console`, `ide disk`, `x86_64`):

	Switch Engine (<code>exosvm.gns3a</code>)	Fabric Engine 9.x (<code>VOSSGNS3-V9.X.gns3a</code>)
Adapters	13 × <code>rtl8139</code> : Mgmt + Port1... Port12	33 × <code>e1000</code> : Mgmt + 1/1... 1/24 + 8 named 1/25/1... 1/26/4
RAM	1024 MB (512 MB also boots)	2048 MB (required since 8.1, 64-bit image)

	Switch Engine (<code>exosvm.gns3a</code>)	Fabric Engine 9.x (<code>V0SSGNS3-V9.X.gns3a</code>)
Default login	admin, no password	rwa / rwa , password change forced at first login
Notes	no GTAC support; VXLAN VTEP and VPLS/VPWS configure but have no data plane; may sit 1-2 min in (pending-AAA) login:	no GTAC support; emulates a 5520-24T with limited data plane (XA1400 equivalent) since 8.10.1

Other useful repos: `EXOS_Apps`, `exoslib` (Python on EXOS), Ansible collections `extreme.exos`, `extreme.voss`, `extreme.fe`. There is **no Site Engine / XIQ-SE image on GitHub**: it comes from the Extreme portal with a license (see `site-engine-in-gns3`). Read the license in each repo: redistribution of the binaries is restricted and publishing test or benchmark results is not allowed.

2. Add a model or version (no GUI, no SSH)

1. Download the qcow2 on the client and check the md5 against the `.gns3a`:

```
curl -O https://akamai-ep.extremenetworks.com/Extreme_P/github-en/Virtual_V0SS/
FEGNS3.9.4.0.0.qcow2
md5sum FEGNS3.9.4.0.0.qcow2
```

2. Upload it and create the template in one go (the script re-checks the md5, refuses to overwrite an existing template name, and prints the new `template_id`):

```
python3 scripts/gns3a_to_template.py V0SSGNS3-V9.X.gns3a --version 9.4.0.0 \
--upload-image FEGNS3.9.4.0.0.qcow2 --post <GNS3_HOST>
```

Without `--post` it prints the JSON only (dry run). Under the hood: `POST /v2/compute/qemu/images/<file>` (raw body) then `POST /v2/templates`. If you have SSH to the server you can instead copy the file to its QEMU images directory (`/opt/gns3/images/QEMU/` on a typical install) and run without `--upload-image`.

3. **Name templates with the version** (EXOS VM 33.6.1.14, FabricEngine VM 9.4.0.0). Never overwrite or edit a template that running labs use; create a new one.
4. Validate in a throwaway project (never on someone's lab): two nodes linked through the **highest** port you intend to use, boot, `show lldp neighbor(s)` on both sides, save the config, and only then use the template for real work.

If you build the template JSON by hand instead of with the script, include `"platform": "x86_64"` (or `qemu_path`); without it the node fails with `qemu-system-None is not found`.

3. Modify a template

`PUT /v2/templates/<template_id>` with the fields to change; nodes created **afterwards** get the new values, existing nodes keep theirs (change a node with `PUT /v2/projects/<p>/nodes/<n> + properties`, node stopped). The script's `--adapters`, `--ram`, `--cpus`, `--console` and `--name` options cover the common cases when creating.

- **QEMU options: keep `-cpu host` on Switch Engine.** It is a QEMU option that passes the host CPU model through to the guest instead of QEMU's generic default model. Without it, the EXOS VM does

not recognise the CPU it runs on and **stops in its bootloader menu** (~> prompt, "c) continue with boot process") waiting for someone to continue the boot by hand. Fabric Engine 9.4.0.0 booted fine without it (tested), but keep it too, as the appliance file does.

- `-enable-kvm` / `-machine accel=kvm` in options are **redundant** on a Linux server with hardware acceleration enabled: GNS3 already adds `-enable-kvm` to every QEMU node (check a node's real `command_line` in the API). The server rewrites the appliance's `-enable-kvm` to `-machine accel=kvm`. Adding it to other templates changes nothing.
- `-nographic`: the switches use the serial console (telnet); harmless either way.
- **More ports on Switch Engine**: adapters 13 → 25 gives Mgmt + Port1...Port24 (`PORT_TO_ADAPTER(n) = n`). Keep one `adapter_type` for the whole template (the official file uses `rtl8139`). Do it in the template, before creating nodes (see the "never resize" rule in `gns3-server-api`). Validate through Port24.
- **Fabric Engine ports**: adapters 1-24 = 1/1...1/24. The appliance names adapters 25-32 as breakout ports 1/25/1...1/26/4, but an unchannelized switch only has 1/25 and 1/26 (40G): **adapter 25 = 1/25, adapter 29 = 1/26**, the other six carry nothing. Rename them in `custom_adapters` if that confuses users.
- `console_type`: keep `telnet` for switches.
- RAM/CPU: do not go below the appliance values without testing boot and your features.

4. Upgrading switches already deployed (without changing template)

- Switch Engine: copy the `.xos` from `Virtual_EXOS` to the switch (only those `.xos` files are valid for the EXOS VM; physical-switch images are not) via TFTP/SCP from a reachable server, e.g. Site Engine's firmware repository), then `install image <file>.xos secondary` → `use image secondary` (or `primary`) → `reboot`, and check with `show switch | include "ver:|Image"`. The config is kept.
- Fabric Engine: in-place upgrade with the release `.tgz` from the CLI (8.1+ requires the 64-bit, 2048 MB settings already in the 9.x appliance). Not validated in our lab.

Environment dependencies — what to look for in yours

Need	What we used	What to identify in yours
Download from GitHub/ akamai	<code>curl</code> on the client	Any HTTP client; proxy settings if needed
Upload images	API upload (<code>--upload-image</code>)	Same, or SCP if you have a shell on the server
Disk space on the server	checked before uploading	Space in the server's images directory (qcow2: 0.25-0.5 GB each)
KVM on the server host	enabled	Without KVM (nested virtualization off) the VMs are very slow or do not boot

Skill switch-engine-in-gns3

Use when operating Extreme Switch Engine (ExtremeXOS/EXOS VM) nodes in a GNS3 lab from Claude Code — first boot and login, the security wizard, saving and backing up configs, default.xsf bootstrap behaviour, TFTP/SNMP for Site Engine, installing .xos images, and Python/UPM requirements. For devices around the switch (LLDP endpoints, 802.1X clients, RADIUS, routers) see simulated-devices-in-gns3.

Assumes a running EXOS VM node on `<GNS3_HOST>` (templates: `extreme-switch-templates`; API and consoles: `gns3-server-api`). Commands go through the node's telnet console with `gns3_console.py` (path: `../gns3-server-api/scripts/`).

Validated environment

- EXOS VM **32.6.3.126** (official template, 13 and 25 adapters), GNS3 2.2.45, Linux client.
- Several multi-switch labs (up to 12 switches per project), saved/versioned configs, `default.xsf` bootstrap from TFTP, SNMP towards Site Engine, `.xos` installation.
- UPM and Python scripts tested with Alpine + lldpd endpoints (see `simulated-devices-in-gns3`).

First boot

1. Boot takes about 1 minute. If the node never reaches `login:` and its console shows a bootloader settings menu (`~>` prompt, "c) continue with boot process"), the template lacks the QEMU option `-cpu host` (see `extreme-switch-templates`). Fix the template or node options (node stopped); `c` at that prompt continues the boot once.
2. The first prompt is `(pending-AAA) login: .` **Logins fail until the switch prints "Authentication Service (AAA) ... is now available"** (Failed to connect to authentication service ... Login incorrect). Wait, don't retry in a loop.
3. Login `admin`, empty password. A security wizard follows (MSTP, telnet, SNMP...). Each question ends with a BEL (`\x07`) and `\r\n` counts as two answers: answer one question at a time, or quit it keeping defaults: `python3 gns3_console.py <GNS3_HOST> <port> --wizard answer q`.
4. `disable clipaging` (the script does not do it for you) before long `show` commands.

Saving and backing up

- `save` asks Do you want to save configuration to primary.cfg? (y/N). With the script: `cmd "save"` then answer `y`. A `*` in front of the prompt (`* SW1.5 #`) means unsaved changes: check it before any reboot or use `image`.
- **Never send a command right after one that may prompt** (`save`, enable ssh key generation, `reboot`, `unconfigure switch`) without checking the prompt came back; the next command is swallowed as the answer.
- Back up several switches at once (saves first, then exports `show configuration`):

```
python3 scripts/backup_configs.py <GNS3_HOST> configs/<session>/v<N> SW1=<port1>
SW2=<port2> --save
```

Keep `configs/` in git, one folder per lab session and version, to restore or diff later.

- `show switch | include "ver:|Image|Config"`: images in each partition, booted image, selected/booted config. Config Booted: Factory Default just means no saved config at the last boot.

default.xsf: what really happens at boot

If the switch has **no saved configuration** (never saved, or config deleted / `unconfigure switch`), it runs `default.xsf` from its filesystem on **every** boot. After the first `save` it boots from the saved config and `default.xsf` no longer runs. Useful to build repeatable lab switches:

- Put the per-switch settings in `default.xsf` (sysName, management IP and route, accounts, SNMP, `tftp get` of lab scripts) and leave the switch unsaved: every reboot returns it to the lab baseline.
- Chicken-and-egg: to `tftp get` the file the switch needs a management IP first, so type it by hand the first time (example with the out-of-band port: `configure vlan Mgmt ipaddress <ip>/<len>` and `configure iproute add default <gw> vr VR-Mgmt`), then `tftp get <tftp_server> vr VR-Mgmt <path>/default.xsf default.xsf`. With in-band management use the VLAN and `vr VR-Default` instead (see next section).
- Anything done by hand (e.g. `enable ssh`) is lost at the next reboot unless you `save`.

EXOS CLI gotchas seen in the lab (32.6.3)

- DHCP on a VLAN: `enable dhcp ipv4 vlan <name>` (`configure vlan <x> ipaddress dhcp` does not exist in this build).
- `enable ipforwarding` globally does not enable it per VLAN: also `enable ipforwarding vlan <name>`.
- Loopback for a test "remote host": `create vlan loop / configure loop ipaddress <ip>/32 / enable loopback-mode vlan loop`.
- Routes must show the `f` flag (in FIB) in `show iproute` to be really active.
- The console session times out and asks for login again after inactivity.
- Verify syntax with `?` on the switch and against the documentation of **your** version (see `extreme-product-docs`): documentation and firmware sometimes disagree.

Management plane for Site Engine and tools

Any IP interface of the switch can be used for management (SSH, SNMP, TFTP/SCP, Site Engine discovery), not only the `Mgmt` port:

- **Out-of-band**: the `Mgmt` port (GNS3 adapter 0), VLAN `Mgmt`, virtual router `VR-Mgmt`. Our labs used this as a design choice: a separate `ethernet_switch` segment linking all `Mgmt` ports with Site Engine, so students can break the data plane without losing management.
- **In-band**: an IP on any data VLAN (e.g. `configure vlan Default ipaddress <ip>/<len>`), in `VR-Default`, reached through the data ports like any other host. Closer to many real networks, and the only option when the `Mgmt` port is not wired.

- The virtual router matters on the switch side: commands that open connections take a `vr` argument (`tftp ... vr VR-Mgmt`, `scp2 vr VR-Default ...`, `ping vr VR-Mgmt ...`, `SNTP/SNMP trap/syslog` targets). Use the VR of the interface you chose. Examples below use `VR-Mgmt`; replace it for in-band management.
- SNMP for discovery: `configure snmpv3 add community "public" name "public" user "v1v2c_ro"` (and `private/v1v2c_rw` for write), `enable snmp access`, `enable snmp access snmp-v1v2c` (and/or `snmpv3` with a user of your choice).
- SNTP: `configure sntp-client primary <ip> vr VR-Mgmt`, `enable sntp-client`.
- SSH: `enable ssh2` (generates a key, prompts; then `save`).

Installing another image (.xos) on a deployed switch

Only the `.xos` files published for the EXOS VM in github.com/extremenetworks/Virtual_EXOS can be installed this way. `.xos` images for physical switch families (Summit, ONIE/5xxx bundles...) are built for other hardware and are not valid for the EXOS VM; to move a node to a version that is not published for the VM there is no in-place path.

1. Put the `.xos` somewhere the switch reaches over its management VR (TFTP server, or SCP from a Linux host such as Site Engine, whose firmware repository is `/tftpboot/firmware/images/`).
2. `download image <tftp_ip> <file>.xos vr VR-Mgmt secondary or scp2 vr VR-Mgmt <user>@<host>:<path>/<file>.xos <file>.xos + install image <file>.xos secondary`.
3. `install` leaves the new partition selected: choose explicitly with `use image primary|secondary`, check that the prompt has no `*`, `reboot`, then `show switch | include "ver:|Image"`. The configuration is kept.
4. The `.xos` files and md5s are in the `Virtual_EXOS README` (32.6.3.126, 32.7.2.19, 33.1.1.31, 33.6.1.14 at the time of writing).

Python and UPM in the lab

- Requirements: `show security python` → Python (current): On; `show security fips-mode` → Off. Python 3 is available on EXOS since 32.2.
- No IP reachability to the switch? Upload a script through the console: `edit script <name>.py` (opens vi) then `gns3_console.py <GNS3_HOST> <port> vi-upload <local file>` (presses `i`, pastes, `ESC`, `:wq`); check with `ls` and `run script <name>.py`. With IP reachability (out-of-band or in-band), `scp2` is simpler.
- To trigger device-detect/UPM events, simulate endpoints with Alpine + Ildpd: `simulated-devices-in-gns3` (includes a known classification difference: the endpoint is seen as `ROUTER`, not `BRIDGE`).

Environment dependencies — what to look for in yours

Need	What we used	What to identify in yours
Console access from scripts	<code>gns3_console.py</code> (Python sockets)	Network access to the console port range
TFTP/SCP source for <code>.xos</code> and <code>default.xsf</code>	a Linux node in the lab (Site Engine or a small VM)	Any host reachable from the switch's management VR
Config versioning	git	Any VCS

Skill fabric-engine-in-gns3

Use when running Extreme Fabric Engine (VOSS VM, 5520-24T emulation) nodes in a GNS3 lab from Claude Code — first boot, forced password change, console quirks, Auto-Sense fabric formation, port-to-adapter mapping (including the 40G ports), saving config, and what has and has not been validated.

Assumes a Fabric Engine template on <GNS3_HOST> (create it from VOSSGNS3-V9.X.gns3a with extreme-switch-templates). Consoles: gns3-server-api.

Validation status — read first

Validated (Fabric Engine 9.4.0.0, GNS3 2.2.45): template from GitHub via the API, boot, first login and password change, Auto-Sense forming an SPBM adjacency between two nodes, LLDP towards Switch Engine, the 40G ports, save config, booting without -cpu host. **Not validated:** services on top of the fabric (L2/L3 VSNS, I-SIDs, DvR, multicast), management IP configuration (out-of-band Mgmt port or in-band), upgrades, Site Engine onboarding of VOSS, anything with more than two nodes. Treat those parts as your own tests, and check the syntax against the Fabric Engine documentation of your release (extreme-product-docs).

First boot

1. Boot to Login: takes about 2-3 minutes. The console is noisy: log lines (ZTP+ discovery, cloud agent trying to reach ExtremeCloud IQ, Auto-Sense) keep scrolling and can land after the Login: prompt.
2. On the console, a first-boot questionnaire may appear (disable MSTP? enhanced security mode? disable telnet? enable SNMPv1/v2c? SNMPv3?). Answer one question at a time; empty answers keep the defaults.
3. Login rwa / rwa. **The first login forces a password change** ("This is an initial attempt using the default password. Please change the password to continue."): enter a new password twice. Drive this one prompt at a time: wait for Password: , Enter the New password : , Re-enter the New password : , then the prompt 5520-24T-FabricEngine:1>:

```
C="python3 ../gns3-server-api/scripts/gns3_console.py <GNS3_HOST> <port>"
$C answer "" --until "Login: ?$"
$C answer rwa --until "Password: ?$"
$C answer rwa --until "New password ?: ?$"
$C answer '<new password>' --until "Re-enter.*: ?$"
$C answer '<new password>' --until "[>#] ?$"
```

4. **Send \r, not \r\n**: with \r\n the switch sees an extra empty line, takes an empty password, and after 3 failures locks the console for 60 seconds ("Maximum number of login attempts reached for console").
5. Then enable (prompt ...:1#) and terminal more disable before long outputs. The session survives disconnecting from the console port:

```
python3 ../gns3-server-api/scripts/gns3_console.py <GNS3_HOST> <port> \
cmd "enable" "terminal more disable" "show sys-info" "show isis adjacencies"
```

What you get with no configuration

- Every data port has **Auto-Sense** enabled. Linking two Fabric Engine nodes port-to-port forms the fabric on its own: the port goes `NNI-ISIS-UP`, `show isis adjacencies` shows the neighbor, an SPBM nickname is auto-generated. Two parallel links go `NNI-MLT`.
- A Switch Engine neighbor is detected as a Fabric Attach proxy (`Auto-Sense port ... entered FA-PROXY-NOAUTH state`), and both see each other by LLDP.
- Useful checks: `show lldp neighbor`, `show lldp neighbor summary`, `show isis adjacencies`, `show interfaces gigabitEthernet auto-sense <ports>`, `show interfaces gigabitEthernet interface <ports>`.
- If you don't want this in a lab (e.g. a port towards a plain host), disable Auto-Sense on that port per the documentation of your release.

Ports and GNS3 adapters

GNS3 adapter	Appliance port name	Switch port
0	Mgmt	management port
1-24	1/1 ... 1/24	1/1 ... 1/24 (1000BaseTX)
25	1/25/1	1/25 (40G)
29	1/26/1	1/26 (40G)
26-28, 30-32	1/25/2-4, 1/26/2-4	no link while the ports are not channelized

The node reports a chassis `5520-24T-FabricEngine` and a dummy serial number.

Saving

`save config` → Save config to file `/intflash/config.cfg` successful. Back up with `backup_configs.py ... --os voss` from the `switch-engine-in-gns3` skill (exports `show running-config`; set `CONSOLE_USER/CONSOLE_PASS` if the console asks for login).

Resources and options

- 2048 MB RAM, 1 vCPU per node (appliance defaults). Several nodes need several GB of RAM on the server: check capacity before building larger fabrics.
- 9.4.0.0 booted fine with and without the QEMU option `-cpu host`; keep it as in the appliance file. `-enable-kvm` from the appliance is redundant (GNS3 adds it).

Environment dependencies — what to look for in yours

Need	What we used	What to identify in yours
Image and template	FEGNS3.9.4.0.0.qcow2 + V0SSGNS3-V9.X.gns3a from GitHub	Same, or the version your lab needs
Interactive console steps	one-prompt-at-a-time Python over sockets	Any expect-like tool, or a human at the console for first login

Skill `simulated-devices-in-gns3`

Use when a GNS3 lab with Extreme switches (Switch Engine/EXOS, Fabric Engine/VOSS) needs other devices connected to the network — LLDP endpoints (APs, phones), 802.1X supplicants and a RADIUS server, routers for OSPF/BGP tests, hosts on tagged VLANs, DHCP/DNS/syslog/TFTP servers, traffic sources — built from lightweight Alpine Linux docker containers. Includes what was tested and what did not work (LAG/LACP from a container).

Real endpoints, servers and routers are rarely available as GNS3 appliances, but a small **Alpine Linux docker container** with the right package covers most needs, costs a few MB of RAM, and boots in a second. Assumes `<GNS3_HOST>` (API: `gns3-server-api`).

Validated environment

Tested with EXOS VM 32.6.3.126 on GNS3 2.2.45 (see `VALIDATION.md`), `alpine:latest` containers, packages from the Alpine repositories:

Device simulated	Package	Status
LLDP endpoint (AP, generic device)	<code>lldpd</code>	Validated (UPM device-detect tests), with a classification caveat, see <code>reference/lldpd_endpoints.md</code>
802.1X supplicant (wired, EAP-MD5)	<code>wpa_supplicant</code>	Validated: authenticated by EXOS netlogin through RADIUS; wrong password → reject, port unauthorized
RADIUS server	<code>freeradius + freeradius-eap</code>	Validated (Access-Accept / Access-Reject to EXOS)
OSPF router	<code>frr</code>	Validated: adjacency <code>FULL</code> with EXOS, loopbacks exchanged and reachable
Host on a tagged VLAN (802.1Q)	<code>busybox ip</code>	Validated: VLAN subinterface ↔ EXOS tagged port
LAG (LACP or static) to the switch	<code>iproute2 bonding</code>	Did not work (see below)
BGP router, DHCP/DNS server, syslog/SNMP trap receiver, TFTP server, traffic generator, LLDP-MED phone, MAC-auth client, PEAP/TLS 802.1X	<code>frr, dnsmasq, rsyslog/net-snmp, tftpd-hpa, iperf3, lldpd</code>	Not validated , recipes below are starting points

Fabric Engine: same containers apply (LLDP was seen between VOSS and EXOS; the rest was not tested against VOSS).

Building blocks

Container node (no template needed; `compute_id` required):

```
POST /v2/projects/<p>/nodes
{"name": "RAD", "node_type": "docker", "compute_id": "local",
 "properties": {"image": "alpine:latest", "adapters": 2, "start_command": "/bin/sh",
                "console_type": "telnet"}}
```

The server pulls `alpine:latest` the first time (it needs Internet access). Or create a reusable template with the same properties (`POST /v2/templates, template_type: docker`).

Pattern: two interfaces per container. `eth0` → an `ethernet_switch` linked to a GNS3 `nat` node, only for installing packages (`udhcpd -i eth0 -q; apk add -q <pkg>`); `eth1` → the switch port under test. This keeps the lab port clean and the switch out of the Internet path. If you prefer no Internet at all, build your own image with the packages preinstalled and push it to a registry the server can reach.

Consoles: the container console (telnet) is a root shell (`/ #`); drive it with `gns3_console.py cmd "..."` like a switch. Addresses: `ip addr add <ip>/<len> dev eth1; ip link set eth1 up`. Do not count on persistence: GNS3 keeps only the directories declared persistent for docker nodes (`extra_volumes; /etc/network` by default) when containers are recreated, so keep the setup commands in a script you can replay (we did not test which changes survive a project close/reopen).

802.1X: supplicant + RADIUS + switch

Topology: RADIUS container `eth1` → switch port 1 (VLAN `radnet`), supplicant `eth1` → port 2.

RADIUS (FreeRADIUS 3 on Alpine):

```
apk add -q freeradius freeradius-eap openssl make
ip addr add 10.0.1.2/24 dev eth1; ip link set eth1 up
printf 'client sw1 {\n ipaddr = 10.0.1.1\n secret = testing123\n}\n' >> /etc/raddb/clients.conf
sed -i 'li testuser Cleartext-Password := "testpass" /etc/raddb/mods-config/files/authorize
cd /etc/raddb/certs && ./bootstrap && chown -R radius:radius /etc/raddb/certs; cd /
(radiusd -X > /tmp/rad.log 2>&1 &) # debug mode, log in /tmp/rad.log
```

The EAP module **does not start without certificates** (`rlm_eap_tls: Failed initializing SSL context`) even if you only use EAP-MD5: run `certs/bootstrap` (self-signed, fine for a lab; also needed for PEAP). Add VSAs (VLAN, Filter-Id) after the password in the `authorize` file to test dynamic VLAN / policy assignment.

Switch Engine (netlogin 802.1X on port 2, RADIUS reached in-band from VR-Default):

```
configure vlan Default delete ports 1
create vlan radnet
configure vlan radnet add ports 1
configure vlan radnet ipaddress 10.0.1.1/24
configure radius netlogin primary server 10.0.1.2 1812 client-ip 10.0.1.1 vr VR-Default shared-
secret testing123
enable radius netlogin
create vlan nlvlan
configure netlogin vlan nlvlan # required first: "Error: No NetLogin VLAN is
configured."
enable netlogin dot1x
enable netlogin ports 2 dot1x
```

Supplicant:

```
apk add -q wpa_supplicant
ip link set eth1 up
printf 'ctrl_interface=/var/run/wpa_supplicant\nap_scan=0\nnetwork={\n key_mgmt=IEEE8021X\n
eap=MD5\n identity="testuser"\n password="testpass"\n eapol_flags=0\n}\n' > /etc/wpa.conf
wpa_supplicant -B -D wired -i eth1 -c /etc/wpa.conf      # Alpine's build has no -f option
wpa_cli -i eth1 status      # EAP state=SUCCESS, suppPortStatus=Authorized
```

Check: `show netlogin port 2` → Yes, Radius 802.1x testuser; `/tmp/rad.log` → Access-Accept.

Negative test: `wpa_cli -i eth1 set_network 0 password "wrong"`, `wpa_cli -i eth1 logoff`; `wpa_cli -i eth1 logon` → EAP state=FAILURE, Access-Reject, switch shows No. EAPOL frames cross GNS3 container links without any special setting. Next steps (not validated): PEAP (eap=PEAP, phase2="auth=MSCHAPV2"), MAC-based auth (enable netlogin mac, RADIUS user = MAC address), several supplicants behind a hub.

Routers for routing tests (FRRouting)

```
apk add -q frr
ip addr add 10.0.3.2/24 dev eth1; ip link set eth1 up; ip addr add 10.99.99.2/32 dev lo
sed -i 's/^ospfd=no/ospfd=yes/' /etc/frr/daemons      # bgpd=yes for BGP
/usr/lib/frr/frrinit.sh start
vtysh -c 'conf t' -c 'router ospf' -c 'ospf router-id 2.2.2.2'\
      -c 'network 10.0.3.0/24 area 0' -c 'network 10.99.99.2/32 area 0' -c 'end'
vtysh -c 'show ip ospf neighbor' -c 'show ip route ospf'
```

Switch Engine side:

```
configure vlan Default delete ports 3
create vlan ospfnet
configure vlan ospfnet add ports 3
configure vlan ospfnet ipaddress 10.0.3.1/24
enable ipforwarding vlan ospfnet
configure ospf routerid 1.1.1.1
configure ospf add vlan ospfnet area 0.0.0.0
enable ospf
show ospf neighbor      # 2.2.2.2 FULL
```

Advertise more prefixes from the container (loopbacks, `ip route add blackhole ... + redistribute static`) to load the switch's routing table. BGP (bgpd=yes, router bgp <asn>, neighbor ...) follows the same pattern: not validated. Useful for: route redistribution, filtering/policies, ECMP (two routers), failover tests, VRRP neighbours (keepalived package, not validated).

Hosts on tagged VLANs

```
ip link add link eth1 name eth1.20 type vlan id 20
ip addr add 10.0.20.2/24 dev eth1.20; ip link set eth1.20 up
```

Switch: `create vlan v20 tag 20`, `configure vlan v20 add ports 3 tagged`, IP on the VLAN. One container can be a host in many VLANs (one subinterface each), handy for inter-VLAN routing, ACL and QoS classification tests.

Other servers and tools (not validated)

- **DHCP/DNS:** `dnsmasq (dhcp-range=..., address=/name/ip);` with the switch as relay (EXOS `configure bootprelay add <server>`), `dnsmasq` answers relayed requests with only a `dhcp-range` for that subnet. Do not use `dnsmasq's own dhcp-relay=` for this.
- **Syslog / SNMP traps:** `rsyslog` or `busybox syslogd -n -0 /dev/stdout`; `net-snmp (snmptrapd -f -Lo)`.
- **TFTP/SCP server** for configs, scripts and `.xos` images: `tftp-hpa (in.tftpd -L -s /srv/tftp)`, `openssh`.
- **Traffic:** `iperf3`, `hping3`, `tcpreplay` to exercise ACLs, QoS, flow export. Mind the limited data plane of the virtual switches: use it for function, not for performance.
- **LLDP-MED phone:** `lldpd` supports MED classes and network policies; check which device type the switch assigns before relying on it.

LAG / LACP from a container: did not work

Tried with a container with two links to EXOS ports 4-5 (`ip link add bond0 type bond mode 802.3ad, iproute2`) and `enable sharing 4 grouping 4-5 lacp`: the container received the switch's LACPDUs (partner MAC set, both members in one aggregator), but EXOS stayed `Selected/Waiting` with aggregator count 0 and no traffic. Static sharing with a `balance-xor` bond failed too, and the link state of ports 4-5 on the EXOS VM flapped between up and down after the container's interfaces were reconfigured. To test LAGs, use two virtual switches back to back (not validated in this pack either).

Why not 802.3x?

802.3x is Ethernet flow control (PAUSE frames). GNS3 links are software tunnels without real congestion or PHY behaviour, so flow control cannot be tested meaningfully; test it on hardware.

Environment dependencies — what to look for in yours

Need	What we used	What to identify in yours
Docker on the GNS3 server	GNS3's docker support, <code>alpine:latest</code> pulled from Docker Hub	Docker installed on the server and a reachable registry
Package installs	NAT node + <code>apk</code>	Internet via NAT, or prebuilt images

Skill `site-engine-in-gns3`

Use when running ExtremeCloud IQ Site Engine (XIQ-SE, formerly Extreme Management Center/NetSight) as a VM inside a GNS3 lab together with virtual Switch Engine/Fabric Engine switches — getting the image in, sizing, disks, console, network placement, discovering switches by SNMP, firmware repository, ZTP+ DNS, MIB Tools, cloning/compacting its disks — and for the (not validated) place of ExtremeControl and ExtremeAnalytics engines.

Assumes `<GNS3_HOST>` (API basics: `gns3-server-api`). Site Engine is **not** on GitHub: it comes from the Extreme support portal and needs a license (an evaluation license works for labs).

Validated environment

- XIQ-SE **24.2.15.5** as a GNS3 QEMU VM on GNS3 2.2.45: web UI, SNMP discovery of EXOS VM switches, firmware repository used to push `.xos` images, FlexViews, MIB Tools.
- Instances at 16 GB RAM / 4 vCPU (as installed) and 8 GB / 2 vCPU (clone used for a single small lab of a few switches). Not validated with Fabric Engine devices.
- Our base VM was an Ubuntu VM with Site Engine installed on it; we did **not** import the vendor virtual appliance ourselves (see "Getting the image in").

Getting the image in

Two routes; pick what your download gives you:

1. **Vendor virtual appliance (OVA/VMDK)** — not validated by us. An `.ova` is a tar archive: `tar xf <file>.ova`, then convert each disk: `qemu-img convert -p -O qcow2 <disk>.vmdk siteengine-hda.qcow2` (and `-hdb` for a second disk). Upload with `POST /v2/compute/qemu/images/<file>` (as `gns3a_to_template.py --upload-image` does) or copy to the server's QEMU images directory, then create a QEMU template with the RAM/CPU/disks from the OVA's `.ovf` descriptor.
2. **Install on a Linux VM** (how our instance was built): a supported Linux QEMU VM in GNS3, then the Site Engine installer inside it following the Installation Guide.

Template settings that worked: `platform: x86_64`, `adapter_type: virtio-net-pci`, 1-2 adapters (see network), disks as `hda/hdb`, `console_type: vnc` (or `spice`), RAM/vCPU as below. Clear options at template level; if you add a QEMU guest-agent socket, set it per node with a unique path (a socket defined in the template makes two instances collide).

Sizing

- Start with what the Installation Guide asks for your version (our install: 16 GB / 4 vCPU).
- 8 GB / 2 vCPU was enough for one small lab (a few switches, one or two users). We did not go lower. Several instances on one server add up quickly: plan RAM before cloning a pool.

Network placement

- Site Engine only needs IP reachability to **some** IP of each switch: the out-of-band Mgmt port or any in-band VLAN interface (see `switch-engine-in-gns3`, "Management plane"). In our labs, by design, Site Engine's NIC sat on the same `ethernet_switch` segment as the switches' Mgmt ports, with a static IP. With in-band management, connect it to a data VLAN (or route to one). On the switch, traffic it originates (traps, syslog, TFTP/SCP) leaves from the VR of the chosen interface.
- For Internet access (licensing, cloud onboarding, updates) add a second NIC to a GNS3 `nat` node (DHCP from `192.168.122.0/24`; that address can change between boots, and it is only reachable from the GNS3 host itself) or route through a Linux router node.
- Web UI: `https://<site_engine_ip>:8443` from any machine with a route to it (e.g. a Linux desktop VM in the lab, reached through its VNC console).
- The console port (VNC/SPICE) can change on every start: read it from the API.

Using it with the virtual switches

- **Discovery**: enable SNMP on the switches (see `switch-engine-in-gns3`), create the matching SNMP credential/profile in Site Engine and discover the management subnet(s) or add devices by IP. Check SNMP from the switch side with `show snmpv3 ...` and from Site Engine with its MIB Tools.
- **Firmware repository**: images uploaded through the web UI (Firmware / image management) land in `/tftpboot/firmware/images/` on the Site Engine server. Switches can pull them by TFTP, or with `scp2 vr <VR-Mgmt|VR-Default> <user>@<site_engine_ip>:/tftpboot/firmware/images/<file>.xos <file>.xos`. For EXOS VM upload only the `.xos` files published in `Virtual_EXOS` (not `.iso`, not images for physical switches).
- **ZTP+**: a switch doing ZTP+ gets an IP and DNS domain by DHCP (on the Mgmt port or an in-band VLAN; check which interfaces your version uses in its User Guide) and resolves `extremecontrol.<domain>` to find Site Engine. In a lab, make that name resolve to your instance: e.g. with `dnsmasq`, `address=/extremecontrol.<domain>/<site_engine_ip>`. If several labs share one flat subnet, tag by client MAC: `dhcp-host=<switch MAC on that interface>,set:lab1,dhcp-option=tag:lab1,15,"lab1.example"`. The MAC of each node port is in the GNS3 API (`ports[].mac_address`). Designed and pre-provisioned in our lab; the full ZTP+ flow was not exercised end to end.
- **MIB Tools** (legacy Java Web Start client): `https://<site_engine_ip>:8443/MIB_Tools.jnlp` (no link in the web UI of 24.2). On a Linux desktop, `icedtea-netx (javaws)` with OpenJDK 11 runs it (accept the self-signed certificate and the signed app; `sun/misc/Launcher` not found traces are harmless). It queries SNMP **from the desktop**, so that desktop needs a route to the switches' management IPs.
- **FlexViews** (custom SNMP table views): skill `site-engine-flexview-builder`.

Cloning and compacting Site Engine disks

To make a reusable template from an instance (or shrink it):

- Compact Site Engine's own database first (from its UI/admin tools), then zero-fill free space inside the guest and `qemu-img convert -O qcow2 -c`. **Zero-filling a thin disk without discard can first grow it massively** (e.g. from 17 GB to 200+ GB) before compaction: check free space first.
- `qemu-img convert` of an image with a backing file **does not always flatten it**: check `qemu-img info <file> | grep backing`. If a backing file remains, `qemu-img rebase -b '' <file> (safe mode)`

merges it; the real size can then be much larger than the first result. Run `qemu-img check` before deleting any intermediate copy.

- Do this work in a large scratch directory on the images filesystem, never in `/tmp` on a small root partition (it filled / once).
- After cloning: new hostname/IP inside the guest, one unique guest-agent socket per node.

ExtremeControl and ExtremeAnalytics — NOT validated

Not deployed in our lab (their virtual appliances were not available). How they would fit, to be confirmed with the product documentation of your version:

- Both are separate engine VMs (Access Control engine for ExtremeControl/NAC, Analytics engine for ExtremeAnalytics) managed from Site Engine. In GNS3 they would be QEMU VMs built from the vendor appliance disks, like route 1 above, each with its own RAM/CPU needs.
- ExtremeControl needs the switches to send RADIUS (802.1X/MAC auth) to the engine: the engine's IP must be reachable from the switches' authentication VR, and test endpoints (Linux VMs with a supplicant, or Alpine containers) must sit behind switch ports.
- ExtremeAnalytics needs flow data (e.g. IPFIX/sFlow) or mirrored traffic from the switches; check which data plane features the virtual switches support (EXOS VM and VOSS VM have a limited data plane).
- If you validate them, add a section here with the same structure (image, sizing, network, gotchas).

Environment dependencies — what to look for in yours

Need	What we used	What to identify in yours
Site Engine image and license	installed on a Linux VM, evaluation-type lab use	Portal access; OVA or installer for your version
Disk conversion	<code>qemu-img</code>	<code>qemu-img</code> on the client or on the server
A browser/desktop inside the lab	Linux desktop VM with Firefox	Any VM with a route to Site Engine, or routing from your own PC

Skill `site-engine-flexview-builder`

Use when creating, editing, or understanding ExtremeCloud IQ Site Engine (XIQ-SE, formerly NetSight/Extreme Management Center) FlexViews (.tpl files) — building a custom SNMP table view from a MIB definition plus a natural-language description of the desired columns, deploying it and reloading the server.

A **FlexView** is an XML definition (`.tpl`) that tells Site Engine which SNMP table to walk and how to show each column in a grid of the UI. There is no official generator from natural language: this skill documents the real format (DTD + examples checked on a Site Engine 24.2.15.5) so one can be written by hand from a MIB file and a description of what should be shown.

Validated environment

- ExtremeCloud IQ Site Engine 24.2.15.5 running as a VM in a GNS3 lab (`site-engine-in-gns3`).
- Access to the Site Engine host by SSH (`ssh <user>@<site_engine_ip>`), `sudo` needed to read and write under the install directory.
- Format details below marked **confirmed** were checked against dozens of factory and GitHub examples; those marked **hypothesis** were not.

Where FlexViews live

The install base is `/opt/Extreme_Networks/NetSight/` on our instance; Extreme's documentation uses `/usr/local/Extreme_Networks/NetSight/`, and it can be chosen at install time. **Check first:** `ls -ld /opt/Extreme_Networks /usr/local/Extreme_Networks 2>/dev/null`; if neither exists, ask the human where it is installed instead of searching blindly. Paths below are relative to `<base>/appdata/`.

Path	Use
<code>System/FlexViews/</code>	Loaded by the running server; organised by category (MAC Locking/, Radius/, Interface/ ...). Also has <code>flextable.dtd</code> .
<code>VendorProfiles/Released/</code> <code><Vendor>/FlexViews/</code>	~400 factory FlexViews (Extreme and other vendors): example library , do not edit.
<code>VendorProfiles/Stage/</code> <code>MyVendorProfile/FlexViews/</code> <code>My FlexViews/</code>	Where custom FlexViews go for Site Engine / Extreme Management Center 8.2+ (create it if missing). <code>System/FlexViews/My FlexViews/</code> was the location for NetSight ≤ 8.1.
<code>VendorProfiles/Stage/</code> <code>MyVendorProfile/MIBs/</code>	The MIB files your custom FlexViews use, so the server resolves symbolic OIDs (create if missing).

More examples: <https://github.com/extremenetworks/XMC-Report-Views> (official `.tpl` examples by category: EXOS/, Networking/, other vendors). Check it before writing one from scratch: a similar view may exist already. Its `FlexView/README.md` documents the path difference by version above.

Reloading the server after adding a FlexView

Copying the file is not enough: restart the Site Engine server service as root (per Extreme's documentation, `systemctl stop nsserver / systemctl start nsserver`; confirm the unit name on your instance with `systemctl list-units | grep -i -E 'nsserver|netsight'`). The restart interrupts the web UI: warn anyone using it first.

XML structure (confirmed by `flextable.dtd`)

```
flextablelist
├─ flextable (id=NAME, class="com.ets.nac.flexview.FvTable", ...)
│   ├── tableModel      → text: com.ets.nac.tables.TbModel
│   ├── column* (id, modelIndex, editable, sortOrder, width, comparator...)
│   │   ├── notes?      → free text, column tooltip
│   │   │   └─ dataField → OID reference (next section)
│   ├── comments?       → general notes of the FlexView
│   └─ dataField?       → TBLHASH:::TBLHDR:... (base MIB table, see below)
```

All attributes of `flextable` and `column` (colours, width, order, editable...) are in the DTD: read `<base>/appdata/System/FlexViews/flextable.dtd` for anything not covered here.

The `dataField` of each column (partly reverse-engineered)

Format: `OID::<name-or-numeric-OID>::SNMP::<numeric block>`. Real example:

```
<dataField>OID::etsysMACLockingLockedAddress::SNMP::SNMPNA;false;4;3;false;false;4;2;0;2;6</dataField>
```

Confirmed:

- The OID can be symbolic (resolved against the MIBs loaded on the server) or the full numeric OID; both appear in factory files.
- First field = access mode: `SNMP` read-only; `SNMPW` editable (SNMP SET; always matches `editable="true"` on the `<column>`); `SNMPNA` hidden/not directly accessible (typically the row key, `hidden="true"`).

Second field = SNMP type (matches ASN.1/BER tags; verify against a real example of the same type before relying on it):

2nd field	SNMP type	Confidence	Seen on
2	INTEGER / enum / TruthValue	Confirmed (several vendors)	<code>extremeImageBooted</code> , <code>ifLinkUpDownTrapEnable</code>
3	BIT STRING	Hypothesis, rare	—
4	OCTET STRING (binary)	Hypothesis	MAC addresses
6	OBJECT IDENTIFIER	Hypothesis	references to another OID
15	DisplayString	Strong hypothesis	<code>sysName</code> , <code>ifName</code> , <code>ifAlias</code> , firmware versions

2nd field	SNMP type	Confidence	Seen on
65	Counter32	Hypothesis	counters
66	Gauge32 / Unsigned32	Hypothesis	values going up and down
67	TimeTicks	Confirmed	sysUpTime
70	Counter64	Hypothesis	high-volume counters

The remaining fields (column position, extra flags, trailing 0;0;0...) are not decoded; they seem to encode the MIB table index position and internal UI flags.

Safest technique: don't build the numeric block from scratch. Find, in the factory library, an existing column whose OID has the **same SNMP SYNTAX** and clone its whole block, changing only the OID name:

```
sudo grep -rl "Counter64" <base>/appdata/VendorProfiles/Released/ | head
```

Reserved columns (always present)

Every FlexView starts with these 3 fixed columns, which are internal metadata, not OIDs. Copy them as-is, with the same `ids` and order, before the real MIB columns:

```
<column id="0" fixed="true" hidden="true">ReqID
  <dataField>OID::ReqID::SNMP::ReqID</dataField></column>
<column id="1" fixed="true" modelIndex="1">IP Address
  <dataField>OID::IP Address::SNMP::IP Address</dataField></column>
<column id="2" fixed="true" hidden="true" modelIndex="2">Instance
  <dataField>OID::Instance::SNMP::Instance</dataField></column>
```

Computed columns (EXPR, advanced, use with care)

Seen in `X0S_firmware_Info.tpl` ("Device Type"): `SNMP::EXPR;true;15;3;false;false;1;2;0;0;0;0` (no `OID::` prefix), plus a table-level suffix `:::GEMHASH:::00000:Device Type::routine 111 DeviceType({ "IP Address":1 } )` calling an internal routine. Not enough evidence to document it: clone a real `EXPR` example instead of inventing the syntax.

Table-level dataField (after the last </column>)

```
<dataField>TBLHASH:::TBLHDR::0;0;true;30;etsysMACLockingStaticStationEntry;V.0.0.2.0;0;0;106</dataField>
```

Names the **base MIB table entry** walked (`XxxEntry`, from the MIB's `SEQUENCE OF`). When the columns come from standard tables Site Engine already knows (`ifTable`, `ifXTable`, indexed by `ifIndex`), `None` works instead of the entry name (`TBLHASH:::TBLHDR::0;0;true;30;None;V.0.3.3.0;0;0;103`). Name the entry explicitly only for new proprietary tables.

Workflow: MIB + natural-language description → .tpl

1. Find the target MIB table from the request and the MIB file (the `SEQUENCE OF XxxEntry` whose columns match).
2. For each wanted column note: OID name, SYNTAX, and `read-only/read-write` (`MAX-ACCESS`).
3. Look for an existing similar view (GitHub repo above, `VendorProfiles/Released/`); adapt it if found. Otherwise clone numeric blocks from columns of the same SNMP type.
4. Write the `.tpl`: the 3 reserved columns, one `<column>` per real OID with a `<notes>` describing the field in plain language, and the final `TBLHASH:::TBLHDR::` with the `Entry` (or `None`).
5. Put it in `VendorProfiles/Stage/MyVendorProfile/FlexViews/My FlexViews/` and the MIB in `VendorProfiles/Stage/MyVendorProfile/MIBs/`.
6. Restart the server service (see above).
7. Check it in the UI: device context menu → **FlexView** → it should be listed; open it on a device that implements the MIB and compare values with an SNMP walk (e.g. MIB Tools).

How it reaches the browser (context only)

The `.tpl` is read by the Java backend (`FlexViewFactory`, `FvTable`, `TbModel`). The web UI uses DWR: `/Monitor/dwr/interface/FlexViewDwr.js`, with calls such as `FlexViewDwr.getFlexViewListByDevices(...)` and `FlexViewDwr.getTableview(query, callback)`. The UI entry point is the device context menu item "FlexView" (`actionId: chooseFlexview`).

Environment dependencies — what to look for in yours

Need	What we used	What to identify in yours
Shell on Site Engine	SSH + <code>sudo</code>	Console or SSH access with root rights
MIB files	vendor MIB bundle for the firmware in use	The MIB package matching your device firmware

Skill `extreme-product-docs`

Use when the user provides (or should provide) Extreme Networks product documentation — Switch Engine/Fabric Engine User Guides, Command References, EMS message catalogs, Site Engine doc collections, release notes, knowledge articles, MIBs — and whenever a CLI syntax, default, log message or feature limit must be confirmed before configuring a switch or Site Engine. Covers preparing the docs folder for search and the rules for using it.

Claude's built-in knowledge of Extreme CLIs is not version-exact, and the documentation itself sometimes disagrees with the firmware. The reliable method: give Claude the official documents **for the exact versions in the lab**, as searchable text, and verify against the live CLI.

Validated environment

- Linux, `pdftotext` (poppler-utils) and Python 3. Used with the Switch Engine 33.1.1 User Guide, Command References and EMS Message Catalog, and a Site Engine documentation collection (zip), while configuring and debugging EXOS switches and UPM/Python scripts.

1. What to provide

Download from the Extreme documentation portal the documents matching the **firmware and software versions in your lab** (`show version` / `show sys-info` / Site Engine "About"):

Product	Most useful documents
Switch Engine (EXOS)	Command References, User Guide, EMS Message Catalog, Release Notes
Fabric Engine (VOSS)	Command Reference, Configuration guides (by feature area), Release Notes
Site Engine (XIQ-SE)	Documentation collection (zip): User Guide, Installation Guide, API docs, Release Notes
ExtremeControl / ExtremeAnalytics	Installation and User Guides of the engine version
Any	Knowledge articles (save the page as HTML), vendor MIB bundle for the firmware

2. Prepare the folder

Put everything in a `docs/` folder of the project (original PDFs/zips/HTML) and run:

```
python3 scripts/prepare_docs.py docs/
```

It unzips collections, converts each PDF with `pdftotext -layout` and each HTML page to `.txt` next to the original, and writes `docs/INDEX.md` (file, product, version, kind, size) with guessed values: **correct the product/version columns by hand**, since the version is what makes a document trustworthy for a given

switch. Re-running only processes new files. Large documents are fine (a Command Reference is ~8 MB of text): they are searched, not read whole.

Tell Claude in the project's `CLAUDE.md` (or at the start of the session) that the folder exists, e.g.: "Official docs for the lab versions are in `docs/`, index in `docs/INDEX.md`. Check syntax there before configuring."

3. Rules for using it (for Claude)

1. **Before stating or sending a command you are not certain of, search the matching document:**
`grep -n -i "<command words>" docs/<command reference>.txt`, then read the section around the hit (`sed -n '<from>,<to>p'`): syntax, defaults, usage guidelines, platform notes.
2. **Match versions:** use the document of the version running on the device; say so when only another version's document is available.
3. **The live CLI wins:** confirm with `?` / tab completion on the device (in a test node, not on a node someone is using). When the document and the firmware disagree, follow the device, and record the discrepancy (e.g. in `docs/DISCREPANCIES.md`) so it isn't rediscovered.
4. **Logs:** look up unknown log messages in the EMS Message Catalog (message name, meaning, severity) before guessing their cause.
5. Cite what you used: "Command Reference 33.1.1, `enable dhcp vlan`, p. 2218", so a human can check it.
6. Quote only what's needed; don't paste large parts of the documents into chats, tickets or public posts (they are copyrighted).

Worked example: lab notes said `enable dhcp ipv4 vlan <name>`; the 33.1.1 Command Reference gives `enable dhcp [ipv4 | ipv6] [vlan_name | all]` (IPv4 is the default when omitted). Both agree, and the device accepted it, so it goes into the skill as confirmed.

Environment dependencies — what to look for in yours

Need	What we used	What to identify in yours
PDF to text	<code>pdftotext</code> (poppler-utils)	poppler for your OS, or any PDF-to-text tool keeping the layout
Search	<code>grep</code>	Any text search tool Claude Code can run
Documents	Extreme documentation portal	Access to the portal for your versions

Appendix: [simulated-devices-in-gns3/reference/lldpd_endpoints.md](#)

The public GNS3 appliance registry ([GNS3/gns3-registry](#)) has **no** IP phone or Wi-Fi access point template (the closest, [cisco-vWLC](#), is a CAPWAP controller, useless as an LLDP endpoint). What worked: an **Alpine container running lldpd** that advertises whatever LLDP identity you need.

Docker template (API v2)

```
POST /v2/templates
{"template_type": "docker", "compute_id": "local", "name": "Alpine-lldpd",
 "image": "alpine:latest", "adapters": 1, "console_type": "telnet", "start_command": "/bin/sh"}
```

Without `"compute_id": "local"` the creation failed on our server. The server needs Internet access (or a local registry) to pull `alpine:latest` the first time.

Preparing the container (through its telnet console)

```
udhcpd -i eth0          # DHCP address (link the container to a NAT node first)
apk update
apk add lldpd
lldpd                   # start the daemon
lldpcli show configuration
```

Access point identity (copy of a real AP)

```
lldpcli configure system description "Vendor AP Software, model-x, Test Lab AP"
lldpcli configure system platform "Vendor AP"
lldpcli configure system hostname "TestAP01"
lldpcli configure system capabilities enabled bridge
lldpcli show chassis details
```

Syntax **not** accepted: `capabilities bridge enabled` and `capabilities bridge, router`. The valid one is `capabilities enabled bridge`. To suppress the Management Address TLV: `lldpcli configure system ip management pattern "nomatch0"`.

Checking from Switch Engine

```
restart port <p>                # forces re-detection (device-detect)
show lldp port <p> neighbors detailed
show upm history detail         # with a UPM profile attached: EVENT.DEVICE
received
show log                        # UPM.Msg.LLDPDevDetected ... device type is
<TYPE><n>
```

Known limitation (unresolved)

With Enabled Capabilities: Bridge, EXOS 32.6.3 classified the lldpd endpoint as **ROUTER**, not BRIDGE, while a real AP with the same capability shows as BRIDGE. Ruled out: an explicit Router capability; the container's ip_forward (sysctl -w net.ipv4.ip_forward=0); the Management Address TLV.

Consequence: to test device-type filters (\$EVENT.DEVICE), temporarily accept ROUTER in the lab and flag it in the code, or use real hardware. lldpd is fine to test the **mechanism** (trigger, arguments, MAC guard, configuration applied and reverted) but does not reproduce EXOS's internal classification exactly.

LLDP-MED phones

lldpd can advertise LLDP-MED (device class, network policies). **Not tested by us** (a real phone was used). If you try it, first check which \$EVENT.DEVICE value EXOS receives.

Appendix: validation log

Evidence behind the "validated" claims in this pack. Figures are functional observations, not performance measurements.

Environment

- GNS3 server 2.2.45 (API v2) on a Linux x86_64 host with KVM, one `local` compute.
- Client: Linux, Python 3 (standard library), `curl`.

Live test: Fabric Engine from GitHub (2026-09-30)

Throwaway project, deleted afterwards. Steps, all through the API from the client:

Step	Result
Download <code>FEGNS3.9.4.0.0.qcow2</code> from the URL in <code>VOSSGNS3-V9.X.gns3a</code>	<code>md5 341aef71e2dbb976e6fa6369b363789e</code> matches the appliance file
<code>gns3a_to_template.py VOSSGNS3-V9.X.gns3a --version 9.4.0.0 --upload-image ... --post <GNS3_HOST></code>	image uploaded via <code>POST /v2/compute/qemu/images/...</code> , template "FabricEngine VM 9.4.0.0" created
2 × Fabric Engine + 1 × Switch Engine (EXOS VM 32.6.3.126) from templates	nodes created with 33 (VOSS) and 13 (EXOS) ports, names from the template
Links FE1 1/1 ↔ FE2 1/1, FE2 1/24 ↔ EXOS Port1	Fabric Engine boots to <code>Login:</code> in about 2-3 min; 1/1 goes Auto-Sense NNI - ISIS - UP (SPBM adjacency) with no configuration; EXOS sees FE2 1/24 by LLDP; FE2 detects EXOS as a Fabric Attach proxy
First Fabric Engine login <code>rwa/rwa</code>	forced password change ("initial attempt using the default password")
<code>save config</code>	<code>Save config to file /intflash/config.cfg</code> successful
Adapters 25 / 29 / 32 (named 1/25/1, 1/26/1, 1/26/4 by the appliance)	25 → VOSS port 1/25 (40G, up, LLDP), 29 → 1/26 (up, LLDP), 32 → no link. Without channelization only adapters 25 and 29 are usable
Node options without <code>-cpu host</code>	Fabric Engine 9.4.0.0 boots normally. Switch Engine 32.6.3.126 stops in its bootloader menu (~> prompt, "c) continue with boot process")
Real QEMU command lines (<code>command_line</code> in the node API)	GNS3 adds <code>-enable-kvm</code> to every QEMU node by itself; the appliance's own <code>-enable-kvm</code> is rewritten to <code>-machine accel=kvm</code> (redundant, no extra effect)
EXOS <code>save</code> then answer <code>y</code> with <code>gns3_console.py</code>	<code>Configuration saved to primary.cfg</code> successfully
Fresh EXOS VM: login during (pending-AAA)	<code>Failed to connect to authentication service ... Login incorrect</code> ; works once "AAA ... is now available" is printed
EXOS first-boot security wizard	quit with answer <code>q --wizard</code> ; defaults kept

Step	Result
<code>backup_configs.py <GNS3_HOST> <dir></code> <code>SW1=<port> --save</code> (separate throwaway EXOS project)	switch saved (prompt loses <code>*</code>), 249-line <code>show</code> configuration written, with the changes made just before
Console line endings	<code>\r\n</code> counts as two lines on both EXOS and VOSS consoles (an empty password was sent); <code>\r</code> works

Live test: simulated devices around Switch Engine (2026-09-30)

Throwaway project, deleted afterwards: EXOS VM 32.6.3.126, a GNS3 `nat` node behind an `ethernet_switch` for package installs, `alpine:latest` containers with 2-3 interfaces.

Test	Result
FreeRADIUS 3 (<code>freeradius</code> , <code>free radius-eap</code>)	fails to start until <code>certs/bootstrap</code> creates the TLS certificates (<code>rlm_eap_tls</code> : Failed initializing SSL context)
EXOS enable <code>netlogin dot1x</code>	Error: No NetLogin VLAN is configured. until configure <code>netlogin vlan <vlan></code>
<code>wpa_supplicant -D wired, EAP-MD5</code> , on EXOS port 2	EAP state=SUCCESS, <code>suppPortStatus=Authorized</code> ; EXOS show <code>netlogin port 2</code> : Yes, Radius 802.1x testuser; RADIUS Access-Accept
Same with a wrong password	EAP state=FAILURE, Access-Reject, EXOS shows No
FRR (<code>frr</code>) OSPF on EXOS port 3	EXOS show <code>ospf neighbor</code> : 2.2.2.2 FULL; each side learns the other's /32 loopback; ping between loopbacks OK
VLAN subinterface <code>eth1.20</code> ↔ EXOS tagged VLAN 20	ping OK
Bond 802.3ad (2 links) ↔ EXOS enable <code>sharing ... lacp</code>	container sees the switch as partner; EXOS stays Selected/Waiting, aggregator count 0, no traffic
Bond <code>balance-xor</code> ↔ EXOS static sharing	no traffic; EXOS ports 4-5 link state flapping (A / R) after container interface changes
<code>wpa_supplicant -f <file></code> on Alpine	option not available in that build (prints usage)

Validated in earlier lab work (2026)

- Switch Engine 32.6.3.126: 13-port official template and a 25-adapter (24 data ports) extension; multi-switch labs with saved configs, `default.xsf`, TFTP, SNMP; `.xos` image install on the secondary partition and switching boot image (32.7.2.19 ↔ 32.6.3.126); UPM/Python scripts with Alpine + Ildpd endpoints.
- Site Engine (XIQ-SE) 24.2.15.5 as a QEMU VM in GNS3: switch discovery by SNMP, firmware repository, FlexViews, MIB Tools, clone/resize/compaction of its disks, 8 GB/2 vCPU and 16 GB/4 vCPU instances.

Not validated

- Simulated devices other than those in the table above (BGP, DHCP/DNS, syslog, traffic tools, PEAP, MAC-auth, LLDP-MED); any of them against Fabric Engine.
- ExtremeControl and ExtremeAnalytics (no OVA available at the time).
- Fabric Engine beyond the steps above (services, SPBM L2/L3 VSNs, DvR, upgrades).
- GNS3 3.x (API v3).

Appendix: scripts in the zip

- `extreme-product-docs/scripts/prepare_docs.py`: Prepare Extreme product documentation so Claude Code can search it.
- `extreme-switch-templates/scripts/gns3a_to_template.py`: Turn a GNS3 appliance file (.gns3a) into a template for a remote GNS3 server (API v2).
- `gns3-server-api/scripts/gns3_console.py`: Talk to the telnet console of a GNS3 node (Switch Engine/EXOS, Fabric Engine/VOSS,
- `switch-engine-in-gns3/scripts/backup_configs.py`: Save and back up the running configuration of several Switch Engine (EXOS) or Fabric

License

Text: CC BY 4.0. Scripts: MIT. See `LICENSE`. Extreme Networks product names and images belong to Extreme Networks; this is a community contribution, not an official Extreme Networks product.

Contributed by Salva Ferrer (AVTN, aquivatunombre.es).